GUÍA DE APRENDIZAJE N° 08: ANÁLISIS ESTÁTICO DEL CÓDIGO Y CALIDAD DE SOFTWARE

1. IDENTIFICACIÓN DE LA GUÍA DE APRENDIZAJE

Programa de Formación	Análisis y Desarrollo de Software (ADSO)	**Código**	228118
Proyecto Formativo	Construcción de una solución informática para el control de procesos organizacionales.		
Fase del Proyecto	Ejecución	**Actividad Proyecto**	AP04 - Construir e integrar módulos de software según requerimientos.
Competencia	220501098 - Evaluar la calidad de los productos de software de acuerdo con estándares de la industria.		
Resultado de Aprendizaje	**593146 - 01** Incorporar actividades de aseguramiento de la calidad del software de acuerdo con estándares de la industria.		
Duración de la Guía	**6 Horas** (Presencial / Trabajo asistido)		

2. PRESENTACIÓN

Estimado aprendiz, en la industria del desarrollo de software moderna, la entrega ágil de aplicaciones no debe comprometer la calidad del código fuente. La calidad interna del software determina directamente su mantenibilidad, escalabilidad y vulnerabilidad ante fallos de seguridad.

El **Análisis Estático de Código** es una técnica fundamental del Aseguramiento de la Calidad (SQA) que permite examinar el código fuente sin necesidad de ejecutarlo. A través de esta práctica, es posible identificar tempranamente fallos potenciales, patrones inseguros, incumplimiento de estándares y vulnerabilidades críticas de manera automatizada.

Esta guía le brindará los conceptos clave y la práctica requerida para integrar herramientas avanzadas como **SonarQube** y **SonarLint** en su flujo de trabajo, adquiriendo habilidades esenciales para auditar, diagnosticar y refactorizar proyectos de software profesional bajo estándares internacionales.

3. FORMULACIÓN DE LAS ACTIVIDADES DE APRENDIZAJE

3.1 Distribución Metodológica del Tiempo (6 Horas)

Fase / Momento	Tiempo	Propósito Dinámico
1. Sensibilización	30 min	Activación de conocimientos previos y contexto práctico sobre deuda técnica.
2. Conceptualización	60 min	Apropiación de conceptos: Code Smells, Complejidad Ciclomática, SonarQube.
3. Demostración	60 min	Demostración guiada por el instructor sobre ejecución e interpretación de análisis estático.
4. Taller Práctico	2 h 30 min	Auditoría y corrección real de un proyecto de código fuente con SonarLint/SonarQube.
5. Socialización	45 min	Revisión entre pares y debate sobre hallazgos y decisiones de refactorización.
6. Cierre	15 min	Evaluación final, síntesis de lecciones aprendidas y cierre institucional.

3.2 Actividad de Reflexión Inicial (30 min)

Participe activamente en el conversatorio de aula respondiendo a las siguientes preguntas detonantes:

- ¿Ha intentado modificar un código antiguo y la corrección de un bug causó dos errores nuevos? ¿Por qué cree que sucede este fenómeno?
- ¿Qué diferencia existe entre un código que funciona "a simple vista" y un código limpio, seguro y mantenible?
- ¿De qué manera el concepto financiero de "deuda" se aplica al desarrollo acelerado de software?

3.3 Actividad de Contextualización y Conceptualización (60 min)

Consulte la documentación y participe en la sesión magistral para asimilar los siguientes conceptos clave:

> **Glosario Técnico de Calidad de Software:**
>
> - **Análisis Estático vs. Dinámico:** El análisis estático inspecciona la estructura del código sin ejecutarlo (sintaxis, reglas, patrones). El dinámico analiza el comportamiento durante la ejecución (pruebas unitarias, carga).
> - **Deuda Técnica (Technical Debt):** El costo acumulado de desarrollo adicional generado por elegir soluciones rápidas e imperfectas en lugar de un diseño óptimo.
> - **Code Smells (Olores de código):** Patrones en la estructura que indican posibles problemas de diseño o mantenibilidad (ej. clases gigantes, métodos extensos, duplicación).
> - **Errores Potenciales (Bugs & Security Hotspots):** Fallos de lógica no detectados por el compilador o brechas de seguridad (SQL Injection, variables no inicializadas, fugas de memoria).
> - **Complejidad Ciclomática:** Métrica de software que mide la cantidad de rutas linealmente independientes a través del fuente (puntos de decisión: `if`, `for`, `while`).

3.4 Demostración Guiada por el Instructor (60 min)

El instructor realizará la ejecución en vivo de un análisis estático de código utilizando un proyecto base (Java / Node.js / C#), demostrando:

1. Instalación y configuración de la extensión **SonarLint** en el IDE de desarrollo (VS Code / IntelliJ).
2. Conexión de la aplicación a un servidor **SonarQube Server / Cloud**.
3. Ejecución del análisis estático inicial y lectura del Dashboard de Métricas de Calidad (Quality Gate).
4. Interpretación de la clasificación de severidad de hallazgos: *Blocker, Critical, Major, Minor, Info*.
5. Proceso de corrección guiada y refactorización paso a paso de un "Code Smell" y una vulnerabilidad crítica.

3.5 Actividad de Transferencia del Conocimiento: Taller Práctico (2 Horas 30 Minutos)

Indicaciones del Taller: En parejas de trabajo, desarrollará la auditoría y refactorización de un repositorio de código provisto por el instructor que contiene fallas intencionadas de calidad y seguridad.

Pasos a Ejecutar en el Taller:

1. **Clonación y Configuración:** Clonar el repositorio asignado e integrar SonarLint en el entorno local.
2. **Escaneo Inicial:** Ejecutar el análisis estático inicial del proyecto e identificar el resumen de métricas:
   - Total de Bugs encontrados.
   - Vulnerabilidades y Security Hotspots.
   - Porcentaje de código duplicado.
   - Estimación de Deuda Técnica (tiempo requerido para arreglar).
3. **Matriz de Hallazgos:** Seleccionar al menos 5 hallazgos críticos (mínimo 2 Code Smells, 2 Bugs y 1 Vulnerabilidad) y documentar su estado inicial.
4. **Refactorización y Corrección:** Modificar el código fuente aplicando buenas prácticas de programación (principios SOLID, clean code) para resolver los problemas reportados.
5. **Validación Final:** Re-escanear el proyecto hasta lograr un reporte con estado **PASSED** en el Quality Gate de SonarQube.

3.6 Socialización y Revisión entre Aprendices (45 min)

Mediante la técnica de *Peer Review* (revisión de pares), cada pareja intercambiará su repositorio refactorizado con otra pareja. Realizarán una coevaluación validando que las soluciones no alteren el funcionamiento funcional de la aplicación y eliminaran efectivamente la deuda técnica reportada.

3.7 Cierre y Evaluación (15 min)

El instructor consolidará las conclusiones generales de la sesión, resolverá inquietudes finales y verificará la entrega formal de la evidencia digital requerida.

4. ACTIVIDADES DE EVALUACIÓN

Evidencias de Aprendizaje	Criterios de Evaluación	Técnicas e Instrumentos de Evaluación
Evidencia de Producto / Desempeño: `Informe de análisis estático con hallazgos y correcciones realizadas.` *El documento PDF debe incluir:* • Capturas del Dashboard inicial (antes de refactorizar). • Tabla detallada de hallazgos (Tipo, Severidad, Archivo, Línea, Descripción). • Evidencia del código refactorizado (Antes vs. Después). • Captura del Dashboard final en verde (Quality Gate Passed).	• Identifica y clasifica correctamente errores de sintaxis, reglas de negocio y vulnerabilidades en el código fuente. • Utiliza herramientas automáticas de análisis estático (SonarQube/SonarLint) según estándares técnicos de SQA. • Aplica técnicas de refactorización para reducir la deuda técnica sin alterar la funcionalidad del software. • Genera informes técnicos de calidad de software con estructura profesional.	**Técnica:** Observación directa y Valoración de producto. **Instrumento:** Lista de Chequeo de Producto (Rubrica de auditoría de código).

5. GLOSARIO DE TÉRMINOS

- **Quality Gate:** Conjunto de condiciones cuantitativas orientadas a métricas que un proyecto debe cumplir antes de considerarse apto para producción.
- **SonarLint:** Extensión para el IDE que proporciona retroalimentación instantánea al desarrollador sobre problemas de código antes de confirmarlos en el repositorio.
- **SonarQube:** Plataforma de evaluación continua de la calidad del código que realiza inspecciones automáticas para detectar errores y brechas de seguridad.
- **Refactorización:** Proceso de reestructurar el código fuente existente sin cambiar su comportamiento externo, mejorando sus atributos no funcionales.

6. REFERENTES BIBLIOGRÁFICOS

- Martin, R. C. (2008). *Clean Code: A Handbook of Agile Software Craftsmanship*. Prentice Hall.
- SonarSource. (2024). *SonarQube Documentation & Rules Glossary*. Recuperado de https://docs.sonarqube.org/
- IEEE. (2014). *IEEE Std 730-2014 - IEEE Standard for Software Quality Assurance Processes*. IEEE Computer Society.

7. CONTROL DEL DOCUMENTO

Nombre	Cargo / Cargo Institucional	Dependencia / Centro	Fecha
Cristian Gonzalez	Instructor de Teleinformática / Programación	Centro de Gestion de Mercados, Logistica y Tecnologias de la Información	Octubre de 2026

Control de Cambios

Versión	Razón del Cambio	Autor	Fecha
1.0	Emisión inicial de la Guía de Aprendizaje N° 08.	Cristian Gonzalez	Octubre de 2026